

I2C Driver Analysis – STM32F446RE

Introduction

The I2C driver is developed to simplify communication using the Inter-Integrated Circuit (I2C) peripheral available in the STM32F446RE microcontroller. Instead of directly accessing I2C registers in the application code, the driver provides a set of APIs that perform all required register-level operations internally. This approach improves code readability, reusability, and maintainability while reducing the complexity involved in configuring and controlling the I2C peripheral.

How the Driver Works

The I2C driver acts as an abstraction layer between the user application and the I2C hardware of the STM32F446RE microcontroller. When the application needs to communicate with an external device over the I2C bus, it interacts with the driver APIs rather than directly modifying hardware registers. During I2C initialization, the application first fills the I2C handle structure with the required communication parameters such as clock speed, device address, acknowledgment control, and Fast Mode duty cycle settings. The I2C_Init() API then reads these parameters and configures the corresponding I2C registers automatically. This design allows different I2C peripherals and communication settings to be configured using the same API without modifying the driver source code. As a result, the driver hides low-level register operations from the application and provides a simple and reusable interface for I2C communication.

For example:

- SCL speed configuration → CCR register
- Rise time configuration → TRISE register
- Device address configuration → OAR1 register
- ACK control configuration → CR1 register
- Start and Stop condition generation → CR1 register
- Data transmission and reception → DR register
- Status monitoring → SR1 and SR2 registers

The application does not need to know which register or bit position must be modified because the driver handles all register-level operations internally.

Hardware Abstraction

Without a driver, configuring an I2C peripheral requires direct register manipulation.

Example:

```
RCC->APB1ENR |= (1<<21);  
I2C1->CR1 |= (1<<0);
```

With the I2C driver:

```
I2C_Handle_t I2C1Handle;
```

```
I2C1Handle.pI2Cx = I2C1;
```

```
I2C1Handle.I2C_Config.I2C_SCLSpeed = I2C_SCL_SPEED_SM;
```

```
I2C1Handle.I2C_Config.I2C_DeviceAddress = 0x61;
```

```
I2C1Handle.I2C_Config.I2C_AckControl = I2C_ACK_ENABLE;
```

```
I2C1Handle.I2C_Config.I2C_FMDutyCycle = I2C_FM_DUTY_2;
```

```
I2C_Init(&I2C1Handle);
```

The I2C driver automatically performs the required peripheral configuration based on the settings provided by the user. In this project, the driver:

- Configures the I2C peripheral for communication.
- Configures the I2C clock speed.
- Configures the device address.
- Enables or disables ACK control.
- Configures Fast Mode duty cycle settings when required.
- Updates the required I2C registers internally through the I2C_Init() API.

This approach hides low-level register manipulation from the application developer and provides a simple API-based interface for I2C configuration and communication.

I2C Driver APIs

The I2C_PeriClockControl() API is used to enable or disable the peripheral clock for an I2C module. Before configuring any I2C peripheral, the corresponding peripheral clock must be enabled. This API ensures that the I2C peripheral receives the required clock from the RCC module.

The I2C_Init() API is responsible for configuring an I2C peripheral according to the parameters specified in the I2C handle structure. It configures communication speed, device address, acknowledgment settings, and Fast Mode duty cycle options. This is the primary API used during I2C initialization.

The I2C_DeInit() API resets the selected I2C peripheral back to its default reset state. This can be useful when reconfiguring the peripheral or restoring default settings.

The I2C_MasterSendData() API is used to transmit data from the master device to a slave device using polling-based communication. The API generates the required start condition, sends the slave address, transmits data bytes, and generates the stop condition when the transfer is completed.

The `I2C_MasterReceiveData()` API is used to receive data from a slave device using polling-based communication. The API monitors the required status flags and retrieves incoming data from the I2C peripheral.

The `I2C_MasterSendDataIT()` API provides interrupt-based data transmission. Instead of continuously polling status flags, the driver uses interrupts to transmit data efficiently while allowing the CPU to perform other tasks.

The `I2C_MasterReceiveDataIT()` API provides interrupt-based data reception. Incoming data is received through interrupt events, improving system responsiveness and reducing CPU utilization.

The `I2C_CloseSendData()` and `I2C_CloseReceiveData()` APIs are used to terminate ongoing interrupt-based communication and restore the peripheral state after completion of data transfer.

The `I2C_SlaveSendData()` API allows an I2C slave device to transmit data to a master device.

The `I2C_SlaveReceiveData()` API allows an I2C slave device to receive data sent by a master device.

The `I2C_IRQInterruptConfig()` API is used to enable or disable I2C interrupt requests within the NVIC controller.

The `I2C_IRQPriorityConfig()` API assigns interrupt priority levels to I2C interrupt sources, allowing proper interrupt management when multiple interrupts occur simultaneously.

The `I2C_EV_IRQHandling()` API handles I2C event-related interrupts such as address matching, data transmission, data reception, and stop condition detection.

The `I2C_ER_IRQHandling()` API handles I2C error-related interrupts including bus errors, arbitration loss, acknowledgment failures, overrun errors, and timeout conditions.

The `I2C_PeripheralControl()` API enables or disables the I2C peripheral after configuration.

The `I2C_GetFlagStatus()` API is used to check the status of I2C flags such as Start Bit, Address Match, Transmit Buffer Empty, Receive Buffer Not Empty, and Transfer Complete conditions.

The `I2C_ManageAcking()` API is used to enable or disable acknowledgment generation during I2C communication.

The `I2C_GenerateStopCondition()` API generates a stop condition on the I2C bus to terminate communication.

The `I2C_SlaveEnableDisableCallbackEvents()` API enables or disables callback events for slave mode communication.

The `I2C_ApplicationEventCallback()` function allows the application layer to receive notification events such as transmission completion, reception completion, stop detection, bus errors, arbitration loss, acknowledgment failures, overrun errors, timeout conditions, data requests, and data reception events.

Practical Example Used in This Project

To test the I2C driver in a real-world application, a BMP280 temperature sensor was interfaced with the STM32F446RE microcontroller using the I2C1 peripheral. The GPIO pins PB8 and PB9 were configured as I2C SCL and SDA lines respectively using the GPIO driver, and the I2C peripheral was initialized through the I2C driver APIs with ACK control enabled and Standard Mode communication speed selected.

After initialization, the BMP280 sensor was configured using the `BMP280_Init()` function. The application continuously read temperature data from the sensor using the `BMP280_ReadTemperature()` function. During this process, the I2C driver managed all communication tasks including start condition generation, slave addressing, data transmission, data reception, acknowledgment handling, and stop condition generation.

The measured temperature value was then converted into a readable string using the `sprintf()` function and transmitted to a serial terminal through USART2 operating at a baud rate of 115200 bps. This allowed the sensor readings to be monitored in real time on a PC terminal.

Successful temperature acquisition from the BMP280 sensor and correct transmission of the measured values through USART communication confirmed that the I2C initialization routines, master transmit and receive operations, acknowledgment management, peripheral control functions, and overall communication mechanisms implemented in the driver were functioning correctly. This project demonstrates the practical use of the I2C driver for interfacing external sensors and acquiring real-time environmental data in embedded systems.

Conclusion

The I2C driver developed for the STM32F446RE microcontroller provides a structured and reusable interface for I2C communication. The driver successfully abstracts low-level hardware operations and allows the application to interact with I2C peripherals through a simple set of APIs. By hiding register-level complexity and providing a modular software architecture, the I2C driver simplifies embedded application development, improves code maintainability, and enables reliable communication with external sensors, displays, and other I2C-compatible devices.